LAPORAN TUGAS BESAR
MACHINE LEARNING — PART 3

SmartClinic No-Show Predictor

Implementasi Model PyTorch (.pth) pada Aplikasi Streamlit

Disusun oleh:

Nama	:	Yudi Ahmad Taqiyyudin
NPM	:	202343501555
Kelas	:	X7J
Program Studi	:	Teknik Informatika
Fakultas	:	Teknik dan Ilmu Komputer
Mata Kuliah	:	Machine Learning
Dosen Pengampu	:	Ahmadi, S.Kom., M.T.I.

UNIVERSITAS INDRAPRASTA PGRI (UNINDRA)

Tahun Akademik 2026/2027

DAFTAR ISI

BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Ketentuan Tugas dari Dosen 1
1.4 Sistematika Laporan 2
BAB II RINGKASAN PART 1–2 DAN PERBEDAAN PART 3 3
2.1 Permasalahan dan Dataset 3
2.2 Preprocessing 3
2.3 Pemodelan dan Hasil Part 2 4
2.4 Perbedaan Implementasi Part 2 dan Part 3 5
BAB III PELATIHAN, EVALUASI, DAN DOWNLOAD MODEL 7
3.1 Arsitektur Model 7
3.2 Konfigurasi Pelatihan 7
3.3 Hasil Evaluasi pada Data Uji 8
3.4 Perbandingan dengan Part 2 8
3.5 Proses Download Model dari Google Colab 9
3.6 Verifikasi Berkas Model 10
BAB IV IMPLEMENTASI APLIKASI 12
4.1 Deskripsi Aplikasi 12
4.2 Teknologi yang Digunakan 12
4.3 Struktur Folder Proyek 13
4.4 Fitur dan Input Aplikasi 13
4.5 Cara Menjalankan Aplikasi di Windows 14
4.6 Tampilan Aplikasi 14
BAB V INTEGRASI MODEL DAN PENGUJIAN 16
5.1 Mekanisme Integrasi Model 16
5.2 Contoh Pengujian Antarmuka 16
5.3 Pengujian Otomatis 17
5.4 Penanganan Input Tidak Valid dan Keterbatasan 18
BAB VI GITHUB REPOSITORY 19
6.1 Informasi Repositori 19
6.2 Ringkasan Isi Repositori 19
6.3 Tampilan Repositori 20
BAB VII PENUTUP 21
7.1 Kesimpulan 21
7.2 Saran 21
DAFTAR PUSTAKA 23

BAB I PENDAHULUAN

1.1 Latar Belakang

Ketidakhadiran pasien pada jadwal konsultasi rawat jalan (*no-show*) menyebabkan slot pelayanan terbuang, waktu tenaga kesehatan tidak termanfaatkan, dan antrean pasien lain menjadi lebih panjang. Apabila kemungkinan ketidakhadiran dapat diperkirakan sejak awal, pihak klinik dapat menyiapkan langkah pendukung, misalnya pengiriman pengingat tambahan atau pengaturan jadwal yang lebih fleksibel.

Pada Tugas Besar Part 1 dirancang gagasan **SmartClinic No-Show Predictor**, yaitu sistem untuk memprediksi ketidakhadiran pasien rawat jalan dengan kerangka kerja CRISP-DM dan dataset publik *Medical Appointment No Shows* dari Kaggle. Pada Part 2 dilakukan *data preprocessing*, pemodelan beserta *tuning* hiperparameter, evaluasi, dan analisis kecocokan model (*good fit*). Part 3 merupakan kelanjutan kedua tahap tersebut, yaitu mewujudkan model menjadi sebuah aplikasi yang dapat digunakan melalui antarmuka pengguna (*User Interface*) dan mendokumentasikannya melalui GitHub.

1.2 Tujuan

Tujuan pengerjaan Tugas Besar Part 3 adalah sebagai berikut.

1. Mengunduh model hasil Google Colab dalam format `.pth`.
2. Membuat aplikasi yang menjalankan tugas *Machine Learning* berdasarkan model tersebut, dengan antarmuka pengguna dan model yang sudah terintegrasi ke dalam *source code*.
3. Mengunggah *source code* aplikasi ke GitHub.
4. Menyusun laporan yang memuat tangkapan layar proses unduh model dan tautan GitHub aplikasi.

1.3 Ketentuan Tugas dari Dosen

Berdasarkan instruksi dosen pengampu, Tugas Besar Part 3 melanjutkan Part 1 dan Part 2 dengan lima kegiatan: (1) mengunduh model hasil Google Colab berformat `.pth`; (2) membuat aplikasi yang menjalankan tugas *Machine Learning*

berdasarkan model yang telah dibuat; (3) membuat akun GitHub; (4) mengunggah *source code* aplikasi ke GitHub; dan (5) mengumpulkan tugas dalam bentuk PDF yang berisi tangkapan layar proses unduh model dan tautan GitHub aplikasi. Target minimal aplikasi adalah memiliki antarmuka pengguna dan model *Machine Learning* yang sudah masuk ke dalam *source code* aplikasi. Penulis telah memiliki akun GitHub sehingga kegiatan (3) tidak dilakukan ulang.

1.4 Sistematika Laporan

Laporan ini disusun dalam tujuh bab. BAB I menguraikan latar belakang, tujuan, dan ketentuan tugas. BAB II meringkas Part 1 dan Part 2 serta menjelaskan perbedaan implementasi pada Part 3. BAB III membahas pelatihan, evaluasi, dan proses unduh model `.pth`. BAB IV menjelaskan implementasi aplikasi. BAB V menjelaskan integrasi model dan hasil pengujian. BAB VI memaparkan repositori GitHub. BAB VII berisi kesimpulan dan saran.

BAB II RINGKASAN PART 1–2 DAN PERBEDAAN PART 3

2.1 Permasalahan dan Dataset

Permasalahan yang diselesaikan adalah **klasifikasi biner**: memprediksi apakah pasien akan hadir (kelas 0) atau tidak hadir (kelas 1, *No-show*) pada jadwal konsultasi. Dataset yang digunakan adalah *Medical Appointment No Shows* dari Kaggle (berkas `KaggleV2-May-2016.csv`) dengan 110.527 baris dan 14 kolom. Dataset tidak memuat nilai kosong maupun baris duplikat. Distribusi target tidak seimbang, yaitu 79,81% pasien hadir dan 20,19% pasien tidak hadir.

2.2 Preprocessing

Tahapan *preprocessing* pada Part 2 dipertahankan tanpa perubahan pada Part 3 agar perbandingan hasil dapat dipertanggungjawabkan. Tahapan tersebut adalah sebagai berikut.

1. Nama kolom diseragamkan (`Hipertension` menjadi `Hypertension`, `Handcap` menjadi `Handicap`, `No-show` menjadi `No_show`).
2. Kolom `ScheduledDay` dan `AppointmentDay` dikonversi menjadi tanggal, kemudian dibentuk tiga fitur turunan: `waiting_days` (selisih hari antara tanggal pendaftaran jadwal dan tanggal janji temu), `appt_weekday` (hari janji temu, 0 = Senin), dan `sched_hour` (jam pendaftaran jadwal).
3. Baris tidak valid dibuang, yaitu `Age` di luar rentang 0 sampai 100 dan `waiting_days` negatif. Sebanyak 13 baris dibuang sehingga tersisa 110.514 baris.
4. Target dibentuk sebagai biner (`Yes` = 1 atau tidak hadir, `No` = 0 atau hadir); kolom identitas dan kolom tanggal mentah dibuang.
5. Data dibagi 80% latih dan 20% uji dengan *stratified split* dan `random_state=42`, menghasilkan 88.411 data latih dan 22.103 data uji (4.462 di antaranya pasien tidak hadir).
6. Fitur numerik distandardisasi (`StandardScaler`), fitur kategorik di-*one-hot encode* dengan penggabungan kategori berfrekuensi kurang dari 100 (`min_frequency=100`), dan fitur biner digunakan apa adanya. Seluruh transformasi hanya di-*fit* pada data latih.

Dua belas fitur yang digunakan model dan perlakuannya diringkas pada Tabel 1. Setelah *preprocessing*, setiap pasien direpresentasikan oleh vektor 92 dimensi (4 fitur numerik, 83 kolom *one-hot*, dan 5 fitur biner).

Tabel 1. Fitur yang digunakan model

Fitur	Jenis	Perlakuan
Gender	Kategorik	One-hot (F, M)
Age	Numerik	StandardScaler; rentang valid 0–100
Neighbourhood	Kategorik	One-hot; 74 kategori dikenal + 1 kolom kategori jarang
Scholarship	Biner	Apa adanya (0/1)
Hypertension	Biner	Apa adanya (0/1)
Diabetes	Biner	Apa adanya (0/1)
Alcoholism	Biner	Apa adanya (0/1)
Handicap	Numerik	StandardScaler; skala 0–4
SMS_received	Biner	Apa adanya (0/1)
waiting_days	Numerik (turunan)	Selisih hari janji temu dan pendaftaran; StandardScaler; nilai valid ≥ 0
appt_weekday	Kategorik (turunan)	Hari janji temu (0 = Senin); one-hot 5 kategori + 1 kolom jarang
sched_hour	Numerik (turunan)	Jam pendaftaran jadwal; StandardScaler

2.3 Pemodelan dan Hasil Part 2

Pada Part 2 dibangun tiga model *scikit-learn*, yaitu *Logistic Regression*, *Decision Tree*, dan *Random Forest*. Hiperparameter dipilih dengan `RandomizedSearchCV` dan validasi silang berstrata (5 lipatan untuk *Logistic Regression* dan *Decision Tree*, 3 lipatan untuk *Random Forest*) dengan kriteria pemilihan F1 pada kelas tidak hadir. Diagnosis kecocokan model memakai selisih ROC-AUC data latih dan data uji: selisih lebih dari 0,05 dikategorikan *overfit*, AUC latih di bawah 0,65 dikategorikan *underfit*, dan selain itu *good fit*.

Tabel 2. Hiperparameter terbaik dan ROC-AUC data uji model Part 2

Model	Hiperparameter terbaik	ROC-AUC uji	Diagnosis
Logistic Regression	C = 0,001; class_weight = balanced	0,6749	Good fit
Decision Tree	max_depth = 7; min_samples_leaf = 200; criterion = entropy; class_weight = balanced	0,7233	Good fit
Random Forest	n_estimators = 200; max_depth = 12; min_samples_leaf = 5; max_features = 0,5; class_weight = balanced	0,7415 (a)	Overfit

(a) Nilai ROC-AUC *Random Forest* pada hasil Colab Part 2 tercatat 0,7414; reproduksi ulang pada tahap Part 3 menghasilkan 0,7415 (selisih 0,0001, kemungkinan akibat perbedaan versi *library*). Nilai *Logistic Regression* dan *Decision Tree* identik dengan Part 2.

2.4 Perbedaan Implementasi Part 2 dan Part 3

Seluruh model Part 2 dibuat dengan *scikit-learn* sehingga tidak dapat disimpan dalam format `.pth` (format *checkpoint* PyTorch). Karena instruksi dosen mensyaratkan berkas `.pth`, Part 3 menggunakan implementasi **PyTorch sebagai pengembangan dari model sebelumnya** untuk memenuhi kebutuhan integrasi aplikasi dan format `.pth`. Model PyTorch pada Part 3 merupakan model baru yang dilatih pada dataset, pembersihan data, pembagian data, dan *preprocessing* yang sama dengan Part 2, sehingga seluruh metrik dievaluasi ulang. Berkas `.pth` tidak diperoleh dengan mengganti ekstensi berkas `.pkl` atau `.joblib`. Perbedaan kedua tahap diringkas pada Tabel 3.

Tabel 3. Perbedaan implementasi Part 2 dan Part 3

Aspek	Part 2	Part 3
Library model	scikit-learn	PyTorch
Model	Logistic Regression, Decision Tree, Random Forest	MLP (Multi-Layer Perceptron)
Penanganan kelas tidak seimbang	class_weight = balanced (hiperparameter)	pos_weight pada BCEWithLogitsLoss (≈ 3,95)
Format penyimpanan	Tidak disimpan sebagai .pth	Checkpoint .pth (bobot, konfigurasi, parameter preprocessing, urutan fitur, pemetaan kelas)
Dataset, pembersihan, split, seed	Medical Appointment No Shows; 80/20 stratified; seed 42	Sama dengan Part 2
Keluaran	Hasil evaluasi dan analisis good fit di notebook	Aplikasi Streamlit dan repositori GitHub

BAB III PELATIHAN, EVALUASI, DAN DOWNLOAD MODEL

3.1 Arsitektur Model

Model yang digunakan adalah *Multi-Layer Perceptron* (MLP) untuk klasifikasi biner dengan struktur berikut: lapisan masukan 92 fitur, lapisan tersembunyi pertama 64 neuron dan kedua 32 neuron (masing-masing diikuti *Batch Normalization*, aktivasi ReLU, dan *Dropout* 0,3), serta satu neuron keluaran berupa *logit*. Probabilitas pasien tidak hadir dihitung dengan fungsi sigmoid terhadap *logit* tersebut, dan kelas ditentukan dengan ambang 0,5 (kelas 0 = Hadir, kelas 1 = Tidak hadir).

3.2 Konfigurasi Pelatihan

Pelatihan dilakukan di Google Colab menggunakan notebook `Tugas_Besar_Machine_Learning_Part3.ipynb`. Sebagian data latih (10%) dipisahkan sebagai *validation set* untuk *early stopping*, sedangkan data uji hanya digunakan pada evaluasi akhir. Konfigurasi pelatihan disajikan pada Tabel 4.

Tabel 4. Konfigurasi pelatihan model MLP

Parameter	Nilai
Random seed	42
Pembagian data	80% latih / 20% uji (stratified); validasi 10% dari data latih (79.569 untuk fit, 8.842 untuk validasi)
Fungsi loss	BCEWithLogitsLoss dengan pos_weight = 3,953 (jumlah hadir / jumlah tidak hadir pada data fit)
Optimizer	Adam, learning rate 0,001, weight decay 0,00001
Batch size	512
Epoch maksimum / early stopping	40 epoch / patience 6 (berdasarkan validation loss)
Epoch terbaik pada model final	34
Framework (Google Colab)	PyTorch 2.14.1 (CPU)

Konfigurasi di atas dipilih dari enam konfigurasi MLP yang dibandingkan pada *validation set* dengan kriteria F1, sejalan dengan kriteria pemilihan pada Part 2.

3.3 Hasil Evaluasi pada Data Uji

Evaluasi dilakukan pada 22.103 data uji dengan kelas positif berupa pasien yang tidak hadir dan ambang keputusan 0,5. Hasilnya disajikan pada Tabel 5 dan Tabel 6. Nilai-nilai ini bersesuaian dengan metrik yang tersimpan di dalam berkas `model_no_show.pth` dan telah dihitung ulang dari model tersebut.

Tabel 5. Metrik evaluasi model PyTorch MLP pada data uji

Metrik	Nilai
Accuracy	0,5976
Precision	0,3097
Recall	0,8079
F1-score	0,4477
ROC-AUC	0,7333
PR-AUC	0,3655
ROC-AUC data latih	0,7610
Selisih AUC latih–uji	0,0278 (Good fit)

Tabel 6. Confusion matrix model PyTorch MLP pada data uji

	Prediksi: Hadir	Prediksi: Tidak hadir	Jumlah
Aktual: Hadir	9.604 (TN)	8.037 (FP)	17.641
Aktual: Tidak hadir	857 (FN)	3.605 (TP)	4.462

Model berhasil mengenali 3.605 dari 4.462 pasien yang sebenarnya tidak hadir (*recall* 80,79%), tetapi 8.037 pasien yang hadir turut diprediksi tidak hadir sehingga *precision* hanya 30,97% dan *accuracy* 59,76%. Pola ini diharapkan karena pembobotan kelas (`pos_weight`) sengaja membuat model lebih sensitif terhadap pasien yang tidak hadir. Selisih AUC latih dan uji sebesar 0,0278 berada di bawah batas 0,05 sehingga model didiagnosis *good fit*.

3.4 Perbandingan dengan Part 2

Perbandingan pada Tabel 7 dilakukan pada data uji yang sama. Model *scikit-learn* dilatih ulang dengan hiperparameter terbaik hasil *tuning* Part 2 pada *pipeline* yang sama.

Tabel 7. Perbandingan model Part 2 dan Part 3 pada data uji

Model	Acc	Prec	Recall	F1	ROC-AUC	Selisih AUC	Diagnosis
Logistic Regression (Part 2)	0,6673	0,3201	0,5764	0,4116	0,6749	−0,0008	Good fit
Decision Tree (Part 2)	0,5632	0,2983	0,8606	0,4430	0,7233	0,0102	Good fit
Random Forest (Part 2)	0,6035	0,3140	0,8138	0,4532	0,7415	0,0572	Overfit
PyTorch MLP (Part 3)	**0,5976**	**0,3097**	**0,8079**	**0,4477**	**0,7333**	**0,0278**	**Good fit**

ROC-AUC model MLP (0,7333) lebih tinggi daripada *Logistic Regression* (0,6749) dan *Decision Tree* (0,7233), serta F1 MLP (0,4477) lebih tinggi daripada kedua model tersebut. *Random Forest* memiliki ROC-AUC dan F1 tertinggi (0,7415 dan 0,4532), tetapi berstatus *overfit* (selisih AUC 0,0572) sehingga tidak layak dipilih menurut kriteria Part 2. Dengan demikian, MLP merupakan model dengan performa terbaik di antara model yang berstatus *good fit*. Selisih ROC-AUC antara MLP dan *Decision Tree* hanya sekitar 0,01, sehingga peningkatannya bersifat tipis dan tidak dapat dinyatakan sebagai perbaikan yang besar.

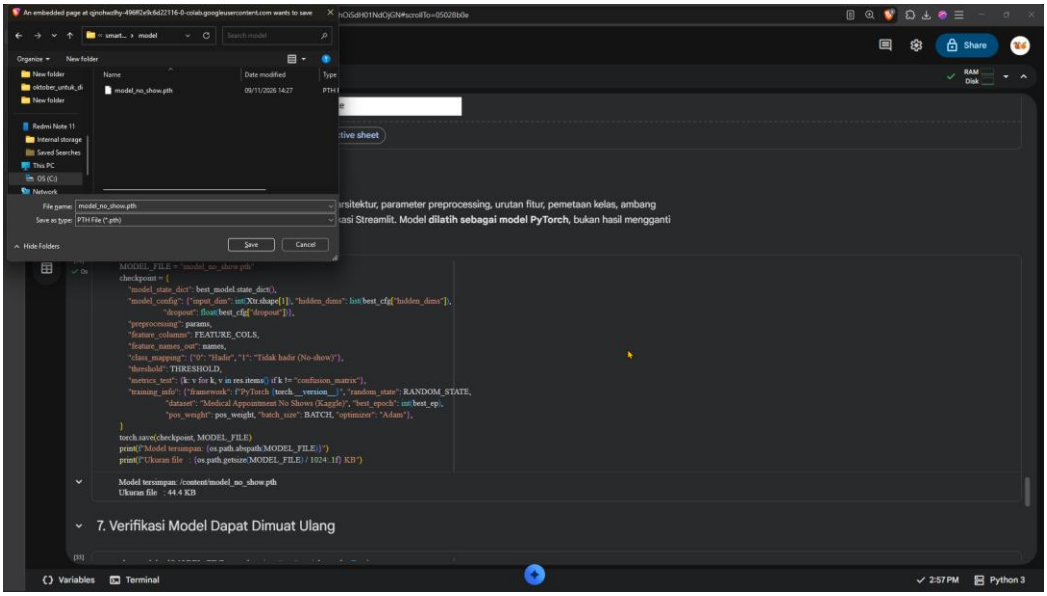
3.5 Proses Download Model dari Google Colab

Setelah pelatihan selesai, model disimpan oleh sel “Simpan Model .pth” pada notebook dengan nama `model_no_show.pth` di direktori kerja Google Colab (`/content/model_no_show.pth`). Keluaran sel tersebut menampilkan ukuran berkas 44,4 KB (Gambar 1 dan Gambar 2). Model kemudian diunduh ke komputer menggunakan perintah berikut pada sel unduh notebook.

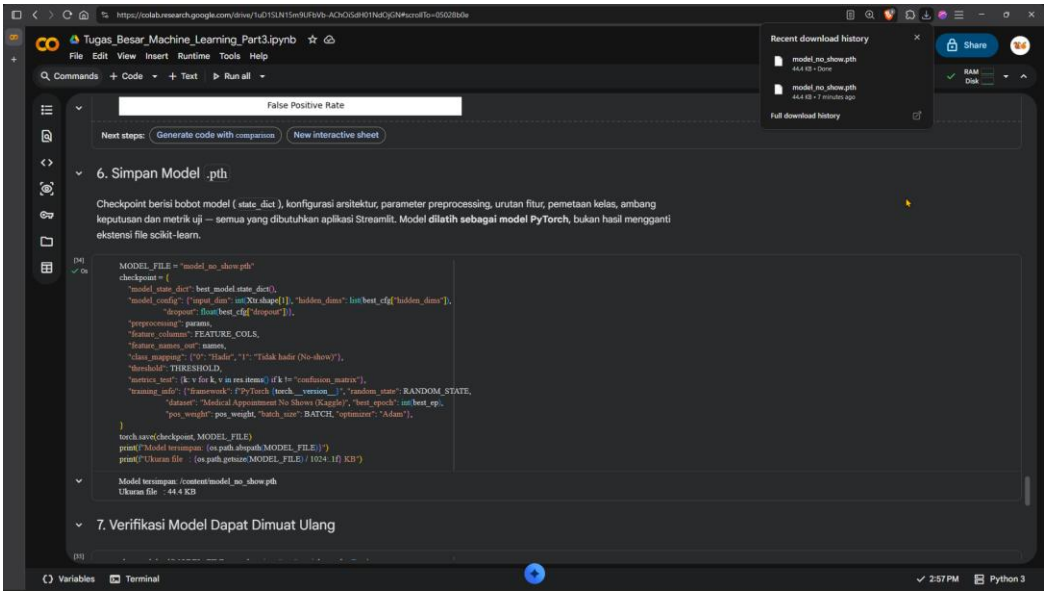
```
from google.colab import files

files.download('model_no_show.pth')
```

Peramban menampilkan dialog penyimpanan berkas dengan nama `model_no_show.pth` bertipe *PTH File (*.pth)*, yang disimpan ke folder `model` pada proyek aplikasi (Gambar 1). Riwayat unduhan peramban mencatat berkas `model_no_show.pth` berukuran 44,4 KB (Gambar 2).



Gambar 1. Dialog penyimpanan berkas model_no_show.pth saat proses download dari Google Colab, dan keluaran sel penyimpanan model (lokasi /content/model_no_show.pth, ukuran 44,4 KB).



Gambar 2. Riwayat unduhan peramban yang mencatat berkas model_no_show.pth berukuran 44,4 KB.

3.6 Verifikasi Berkas Model

Informasi berkas model yang dihasilkan disajikan pada Tabel 8. Verifikasi dilakukan dengan memuat ulang model dari berkas pada repositori menggunakan `torch.load(..., weights_only=True)`, membangun ulang arsitektur MLP dari konfigurasi yang tersimpan, lalu menghitung ulang ROC-AUC dan F1 pada data uji. Hasilnya sama dengan nilai yang tersimpan di dalam *checkpoint* (ROC-

AUC 0,7333; F1 0,4477), sehingga model dapat dimuat kembali dan menghasilkan prediksi yang konsisten.

Tabel 8. Informasi berkas model

Informasi	Keterangan
Nama berkas	model_no_show.pth
Lokasi di Google Colab	/content/model_no_show.pth
Lokasi pada proyek	model/model_no_show.pth
Ukuran	44,4 KB (45.431 byte pada repositori)
Isi checkpoint	model_state_dict, model_config (input 92; hidden 64 dan 32; dropout 0,3), preprocessing (rata-rata/skala, daftar kategori), feature_columns, feature_names_out, class_mapping, threshold (0,5), metrics_test, training_info
Pemetaan kelas	0 = Hadir; 1 = Tidak hadir (No-show)

BAB IV IMPLEMENTASI APLIKASI

4.1 Deskripsi Aplikasi

Aplikasi **SmartClinic No-Show Predictor** merupakan aplikasi web berbasis Streamlit yang memperkirakan kemungkinan pasien hadir atau tidak hadir pada jadwal konsultasi. Pengguna mengisi data pasien dan jadwal pada formulir, kemudian aplikasi menampilkan estimasi probabilitas hadir dan tidak hadir beserta kelas prediksinya. Seluruh probabilitas berasal dari model PyTorch pada berkas `model_no_show.pth`, bukan dari prediksi statis atau acak.

4.2 Teknologi yang Digunakan

Tabel 9. Teknologi yang digunakan

Teknologi	Fungsi
Python 3	Bahasa pemrograman utama
PyTorch	Pelatihan dan inferensi model MLP; penyimpanan dan pemuatan checkpoint .pth
Streamlit	Antarmuka pengguna aplikasi web
pandas dan NumPy	Pengolahan data dan komputasi numerik
scikit-learn	Pembagian data, ColumnTransformer, metrik evaluasi, dan model Part 2 (pelatihan dan pengujian)
pytest	Pengujian otomatis
Google Colab	Pelatihan model dan pembuatan berkas .pth
Git dan GitHub	Pengelolaan versi dan publikasi source code

4.3 Struktur Folder Proyek

Struktur folder proyek pada repositori adalah sebagai berikut.

```
smartclinic-no-show-predictor/
├── app.py                    # aplikasi Streamlit
├── model/
│   └── model_no_show.pth     # checkpoint PyTorch
├── src/
│   ├── preprocessing.py
│   ├── model_architecture.py
│   └── prediction.py
├── notebooks/
│   └── Tugas_Besar_Machine_Learning_Part3.ipynb
├── scripts/                  # train, verify_model, make_report
├── tests/                    # test_prediction, test_app
├── reports/                  # perbandingan dan evaluasi
├── data/                     # README.md (CSV tidak diunggah)
├── assets/  dan  docs/screenshots/
├── .streamlit/config.toml
├── requirements.txt  dan  requirements-train.txt
├── README.md
└── .gitignore
```

4.4 Fitur dan Input Aplikasi

Aplikasi menyediakan halaman utama dengan nama aplikasi dan penjelasan fungsi, formulir input dua belas fitur, tombol *Prediksi*, tampilan kelas prediksi beserta probabilitas hadir dan tidak hadir (bilah proporsi dan angka), panel “Fitur yang dikirim ke model”, tombol *Hapus / ulangi input*, halaman “Tentang Model”, serta keterangan bahwa aplikasi merupakan proyek akademik. Elemen formulir dan pemetaannya ke fitur model disajikan pada Tabel 10.

Tabel 10. Input aplikasi dan pemetaannya ke fitur model

Input pada aplikasi	Fitur model	Keterangan
Jenis kelamin	Gender	Perempuan (F) atau Laki-laki (M)
Umur (tahun)	Age	Bilangan bulat 0–100
Kelurahan	Neighbourhood	Daftar kategori dari checkpoint + opsi “Lainnya (kategori jarang)”
Penerima beasiswa	Scholarship	Kotak centang (0/1)
Hipertensi / Diabetes / Alkoholisme	Hypertension / Diabetes / Alcoholism	Kotak centang (0/1)
Tingkat disabilitas	Handicap	Pilihan 0–4
SMS pengingat terkirim	SMS_received	Kotak centang (0/1)
Tanggal pendaftaran jadwal	waiting_days	Dihitung otomatis: tanggal janji temu dikurangi tanggal pendaftaran
Tanggal janji temu	waiting_days, appt_weekday	appt_weekday dihitung otomatis (0 = Senin)
Waktu pendaftaran jadwal	sched_hour	Hanya jam yang digunakan

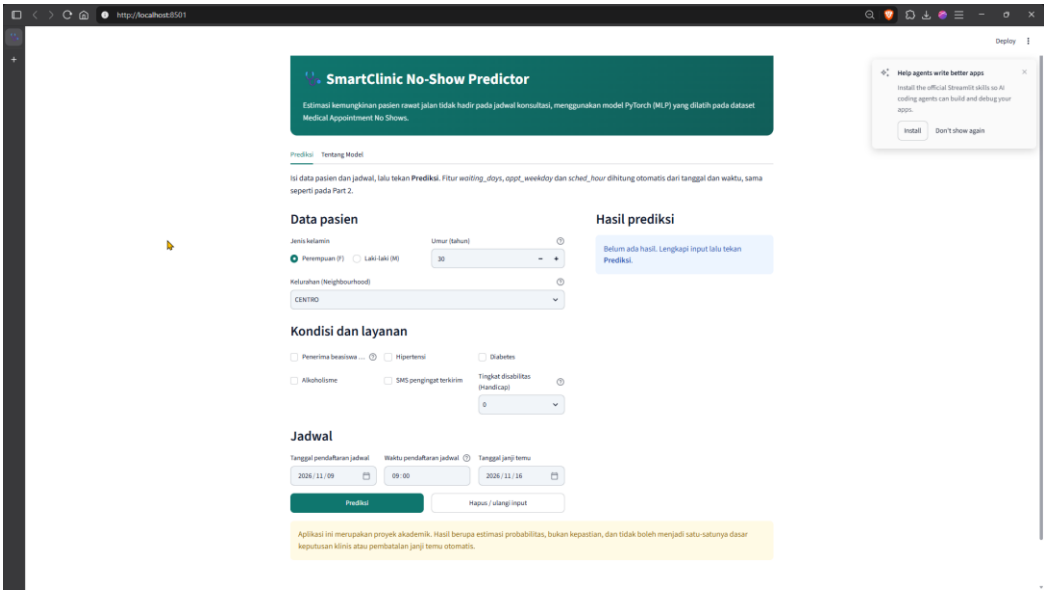
4.5 Cara Menjalankan Aplikasi di Windows

Aplikasi dijalankan dari folder proyek menggunakan lingkungan virtual Python dengan perintah berikut (PowerShell). Peramban akan membuka alamat `http://localhost:8501`.

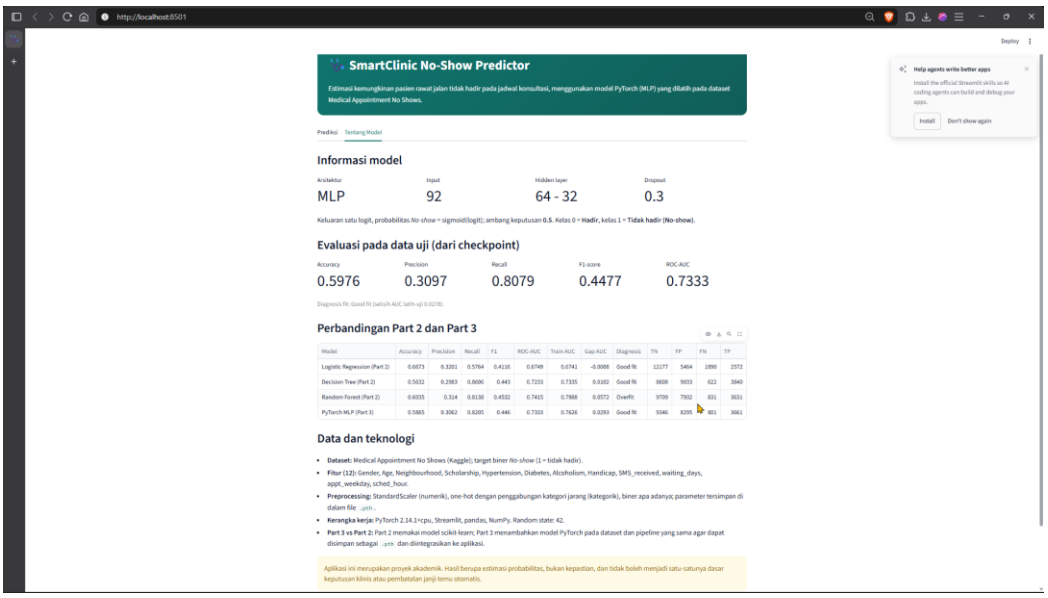
```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
streamlit run app.py
```

4.6 Tampilan Aplikasi

Gambar 3 menampilkan halaman awal aplikasi sebelum prediksi dijalankan, sedangkan Gambar 4 menampilkan halaman “Tentang Model” yang memuat informasi arsitektur, metrik evaluasi yang dibaca dari *checkpoint*, serta tabel perbandingan Part 2 dan Part 3.



Gambar 3. Halaman awal aplikasi SmartClinic No-Show Predictor (alamat http://localhost:8501).



Gambar 4. Halaman Tentang Model: arsitektur MLP (input 92, hidden 64–32, dropout 0,3), metrik data uji dari checkpoint, dan tabel perbandingan.

Catatan: baris "PyTorch MLP (Part 3)" pada tabel perbandingan di Gambar 4 bersumber dari berkas reports/comparison_part2_vs_part3.csv hasil pelatihan awal di lingkungan pengembangan lokal (accuracy 0,5885; F1 0,4460), sehingga sedikit berbeda dari metrik di bagian atas gambar yang dibaca dari model final hasil Google Colab. Nilai resmi laporan ini adalah nilai model final hasil Google Colab (Tabel 5 dan Tabel 7).

BAB V INTEGRASI MODEL DAN PENGUJIAN

5.1 Mekanisme Integrasi Model

Integrasi model `.pth` ke dalam aplikasi mengikuti alur berikut.

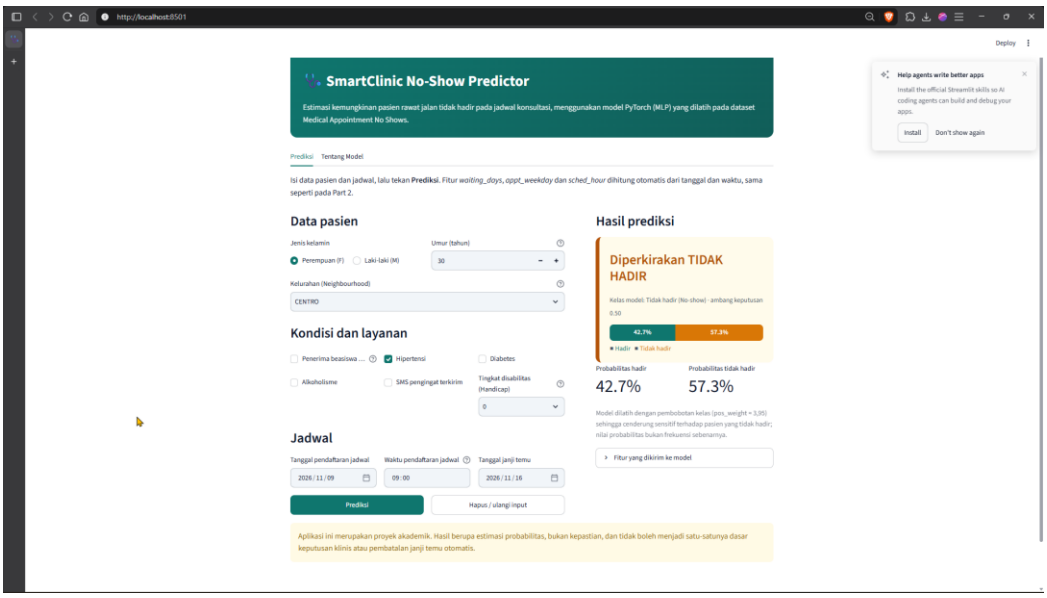
1. Aplikasi memuat berkas `model/model_no_show.pth` dengan `torch.load(..., weights_only=True)`, memeriksa kelengkapan kunci *checkpoint*, urutan fitur, dan kesesuaian dimensi masukan (92), lalu membangun arsitektur MLP dari konfigurasi yang tersimpan dan memuat bobotnya.
2. Model dijalankan dalam mode evaluasi (`model.eval()`), dan prediksi dilakukan dengan `torch.inference_mode()` sehingga aplikasi tidak melakukan pelatihan ulang.
3. Input pengguna divalidasi, lalu fitur `waiting_days`, `appt_weekday`, dan `sched_hour` dihitung dengan aturan yang sama seperti Part 2.
4. Dua belas fitur disusun sesuai urutan pelatihan, kemudian ditransformasi menjadi vektor 92 dimensi menggunakan parameter *scaler* dan *encoder* yang tersimpan di dalam *checkpoint*. Kategori yang jarang atau tidak dikenal dipetakan ke kolom kategori jarang, sebagaimana pada pelatihan.
5. Keluaran model berupa *logit* diubah menjadi probabilitas tidak hadir dengan sigmoid; probabilitas hadir adalah 1 dikurangi nilai tersebut. Kelas prediksi ditentukan dengan ambang 0,5.

5.2 Contoh Pengujian Antarmuka

Pengujian antarmuka dilakukan dengan menjalankan aplikasi pada komputer lokal. Contoh input pada Tabel 11 menghasilkan probabilitas hadir 42,7% dan probabilitas tidak hadir 57,3%, sehingga aplikasi menampilkan kelas “Diperkirakan TIDAK HADIR” (Gambar 5). Nilai yang sama diperoleh ketika model pada repositori dipanggil langsung melalui fungsi prediksi dengan input yang identik.

Tabel 11. Contoh input pengujian

Input	Nilai
Jenis kelamin / Umur	Perempuan (F) / 30 tahun
Kelurahan	CENTRO
Kondisi dan layanan	Hipertensi dicentang; kondisi lain dan SMS pengingat tidak dicentang; Handicap 0
Pendaftaran jadwal	9 November 2026, pukul 09.00
Janji temu	16 November 2026
Fitur turunan (otomatis)	waiting_days = 7; appt_weekday = 0 (Senin); sched_hour = 9
Hasil	Hadir 42,7%; Tidak hadir 57,3% → Diperkirakan tidak hadir (ambang 0,50)



Gambar 5. Hasil prediksi aplikasi untuk contoh input pada Tabel 11.

5.3 Pengujian Otomatis

Pengujian otomatis dengan `pytest` dijalankan pada salinan repositori yang diunduh dari GitHub (dengan dataset tersedia). Seluruh 23 uji dinyatakan lulus. Ringkasannya disajikan pada Tabel 12.

Tabel 12. Ringkasan pengujian otomatis

Kelompok pengujian	Jumlah uji	Yang diperiksa	Hasil
Pemuatan checkpoint dan struktur	5	Kunci dan konfigurasi checkpoint (92 fitur, hidden 64–32, dropout 0,3, ambang 0,5, pemetaan kelas), model berada pada mode eval, konsistensi muat ulang, checkpoint rusak atau tidak ada ditolak	Lulus
Preprocessing	4	Fitur waktu sesuai Part 2, bentuk vektor dan one-hot, kategori jarang masuk kolom infrequent, daftar kelurahan	Lulus
Prediksi	2	Probabilitas dalam rentang 0–1 dan sama dengan keluaran model; hasil berbeda untuk input berbeda (tidak statis)	Lulus
Validasi input	6	Umur, handicap, jenis kelamin, dan tanggal tidak valid ditolak; peringatan untuk data di luar rentang pelatihan	Lulus
Reproduksi metrik	1	ROC-AUC dan F1 data uji dihitung ulang dari .pth sama dengan nilai tersimpan	Lulus
Antarmuka Streamlit	5	Halaman awal, prediksi, penolakan tanggal tidak valid, tombol hapus, dan perubahan hasil terhadap input	Lulus
Total	**23**		**23 lulus**

5.4 Penanganan Input Tidak Valid dan Keterbatasan

Aplikasi menolak input yang tidak valid, yaitu umur di luar 0–100 tahun, nilai *handicap* di luar 0–4, dan tanggal janji temu yang mendahului tanggal pendaftaran jadwal (data seperti ini dibuang pada Part 2). Aplikasi menampilkan peringatan apabila jeda melebihi 179 hari (batas maksimum data latih) atau janji temu jatuh pada hari Sabtu atau Minggu (data latih hanya memuat 39 baris Sabtu dan tidak memuat hari Minggu), karena estimasi pada kondisi tersebut kurang andal.

Beberapa keterbatasan perlu dicatat. *Precision* model rendah (30,97%) sehingga banyak pasien yang hadir turut diprediksi tidak hadir. Dataset berasal dari data tahun 2016 di Brasil sehingga belum tentu mewakili kondisi klinik lain. Probabilitas yang ditampilkan merupakan estimasi dari model yang dilatih dengan pembobotan kelas, bukan frekuensi sebenarnya. Aplikasi ini merupakan proyek akademik dan hasilnya bukan dasar tunggal untuk keputusan klinis atau pembatalan janji temu secara otomatis.

BAB VI GITHUB REPOSITORY

6.1 Informasi Repositori

Source code aplikasi, berkas model `.pth`, notebook Google Colab, dokumentasi, dan pengujian telah diunggah ke GitHub menggunakan akun yang telah dimiliki penulis. Informasi repositori disajikan pada Tabel 13.

Tabel 13. Informasi repositori GitHub

Informasi	Keterangan
Username GitHub	yudiahmadtaqiyyudin
Nama repositori	smartclinic-no-show-predictor
Visibilitas	Public
Branch	main
Jumlah commit	2
Berkas model	model/model_no_show.pth (model final hasil Google Colab)
Tautan repositori	https://github.com/yudiahmadtaqiyyudin/smartclinic-no-show-predictor

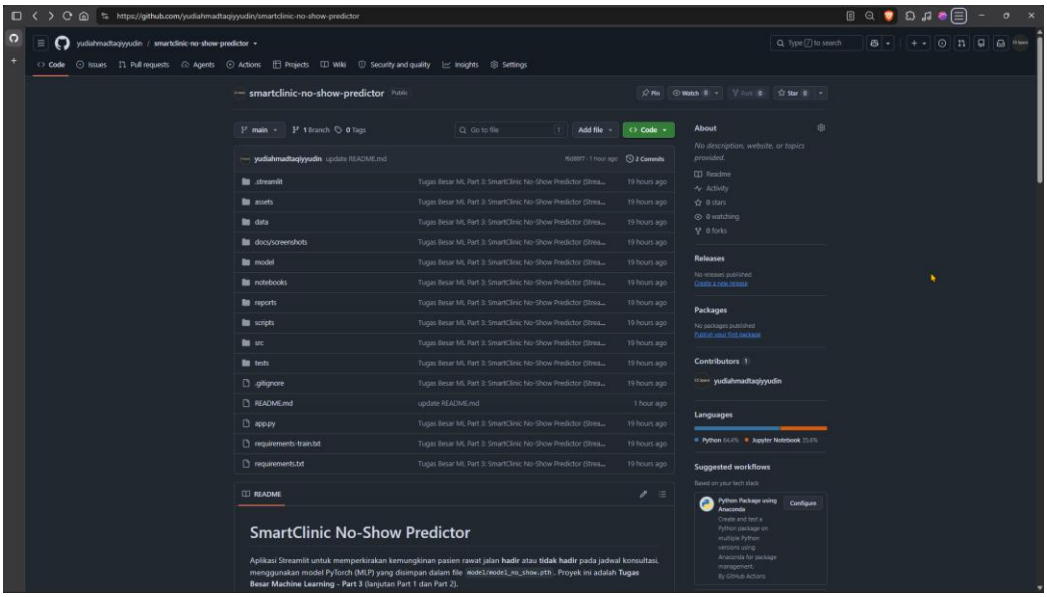
6.2 Ringkasan Isi Repositori

Tabel 14. Ringkasan isi repositori

Berkas / folder	Keterangan
app.py	Aplikasi Streamlit SmartClinic No-Show Predictor
model/model_no_show.pth	Checkpoint model PyTorch MLP (bobot, konfigurasi, preprocessing, pemetaan kelas)
src/	preprocessing.py, model_architecture.py, prediction.py
notebooks/	Tugas_Besar_Machine_Learning_Part3.ipynb (notebook Google Colab)
scripts/	train.py, verify_model.py, make_report.py
tests/	test_prediction.py, test_app.py, conftest.py (23 uji)
reports/	comparison_part2_vs_part3.csv dan folder evaluation/ (metrik dan grafik)
README.md	Deskripsi proyek, instalasi, dan cara menjalankan aplikasi
requirements.txt, requirements-train.txt	Daftar dependensi aplikasi dan dependensi pelatihan
.gitignore, .streamlit/, data/, assets/, docs/	Konfigurasi git dan tema; dataset CSV tidak diunggah (lihat data/README.md)

Catatan: berkas pada folder reports/ (comparison_part2_vs_part3.csv dan evaluation/) dihasilkan pada pelatihan awal di lingkungan pengembangan lokal dengan hasil yang sedikit berbeda (ROC-AUC 0,7333; accuracy 0,5885; F1 0,4460). Model final yang dipakai aplikasi dan dilaporkan pada laporan ini adalah model hasil Google Colab pada berkas model/model_no_show.pth.

6.3 Tampilan Repositori



Gambar 6. Halaman repositori GitHub smartclinic-no-show-predictor (Public, branch main) beserta berkas README.md.

BAB VII PENUTUP

7.1 Kesimpulan

1. Model PyTorch MLP berhasil dilatih di Google Colab pada dataset *Medical Appointment No Shows* dengan *preprocessing* dan pembagian data yang sama seperti Part 2, disimpan sebagai `model_no_show.pth` (44,4 KB), diunduh ke komputer, dan terbukti dapat dimuat ulang dengan hasil yang konsisten.
2. Pada data uji, model mencapai ROC-AUC 0,7333, F1-score 0,4477, *recall* 0,8079, *precision* 0,3097, dan *accuracy* 0,5976 dengan diagnosis *good fit*. Performanya lebih baik daripada *Logistic Regression* dan *Decision Tree* Part 2 (peningkatan tipis) dan sedikit di bawah *Random Forest* yang berstatus *overfit*.
3. Aplikasi Streamlit SmartClinic No-Show Predictor berhasil dibuat dengan antarmuka pengguna, formulir dua belas fitur, perhitungan otomatis fitur waktu, penampilan probabilitas, tombol hapus input, dan penanganan input tidak valid. Seluruh prediksi berasal dari model `.pth`, dan 23 pengujian otomatis dinyatakan lulus.
4. Source code, model, notebook, dan dokumentasi telah diunggah ke repositori GitHub publik sehingga dapat diakses melalui tautan yang tercantum pada BAB VI.
5. Part 3 menggunakan implementasi PyTorch sebagai pengembangan dari model Part 2 untuk memenuhi kebutuhan format `.pth` dan integrasi aplikasi; model PyTorch merupakan model baru sehingga seluruh metrik dievaluasi ulang.

7.2 Saran

1. Menyetel ambang keputusan (*threshold*) menggunakan validation set sesuai kebutuhan klinik, misalnya menyeimbangkan *recall* dan *precision*.
2. Menambahkan fitur yang relevan (misalnya riwayat ketidakhadiran pasien) dan melakukan kalibrasi probabilitas.

3. Melakukan validasi menggunakan data klinik yang sesungguhnya sebelum digunakan dalam pengambilan keputusan operasional.
4. Mempertimbangkan penerapan aplikasi pada layanan *hosting* agar dapat diakses tanpa instalasi lokal.

DAFTAR PUSTAKA

Kaggle. *Medical Appointment No Shows* (berkas KaggleV2-May-2016.csv). Dataset publik pada platform Kaggle.

Paszke, A., dkk. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems 32*.

Pedregosa, F., dkk. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Streamlit Inc. *Streamlit Documentation*. Dokumentasi resmi Streamlit.